{JS} MASTERY

FIELD GUIDE

Claude Code Practical Guide

A practical field guide to the Claude Code commands, shortcuts, files, workflows, and advanced agent features you'll actually use while building real software.

VERSION-DEPENDENT

Claude Code changes quickly, so treat specific commands and experimental features as version-dependent. Type / inside your own Claude Code session to see exactly what's available to you. Anthropic also notes that some commands depend on your platform, plan, and environment.

Starting Claude Code

Start a normal interactive session

TERMINAL

```
claude
```

Then, ask it do anything,

> → Explore this project and explain how product data currently flows from the database to the storefront.

Start Claude with the first task already provided

```
claude "your task"
```

Example,

```
claude "find out why the inventory tests are failing"
```

Useful when you already know exactly what you want Claude to start with.

Run one task and exit

```
claude -p "your task"
```

Example,

```
claude -p "review the current git diff for obvious regressions"
```

Claude performs the task, outputs the result, and exits.

USE IT WHEN

You want Claude inside:

- shell scripts
- CI pipelines
- automated checks
- cron jobs
- command pipelines
- other developer tools

You can pipe information into it too:

```
tail -n 300 app.log | claude -p "find the likely cause of these errors"
```

Continue your latest session

```
claude -c
```

Use this when you closed Claude Code but want to continue the most recent conversation in the current project.

Resume a specific older session

```
claude -r
```

or:

```
claude -r "auth-refactor"
```

Inside Claude Code:

```
/resume
```

Use this when you want to return to a particular piece of work instead of starting fresh.

> Claude Code stores sessions separately, and resuming continues the existing session rather than copying it.

The Commands You'll Use Constantly

COMMAND	WHAT IT DOES	USE IT WHEN
/help	Shows commands available to you	You forget a command
/clear	Clears current conversation context	Moving to unrelated work
/compact	Compresses current conversation	Same task, context getting large
/context	Shows where context is being used	Claude starts losing track
/memory	Inspect/edit Claude memory and instructions	You want to see what Claude remembers
/model	Switch model	Task needs a different capability level
/effort	Change reasoning effort	Hard debugging or architecture
/permissions	Manage what Claude can do	Control access/actions
/plan	Enter Plan Mode	Approach needs review first
/diff	Review changes Claude made	Before accepting or shipping

COMMAND	WHAT IT DOES	USE IT WHEN
`/review`	Review code without immediately editing	Before merging
`/security-review`	Security-focused review	Auth, secrets, APIs, permissions
`/rewind`	Return to an earlier checkpoint	Claude went down the wrong path
`/doctor`	Diagnose Claude Code	Installation/config feels broken
`/tasks`	Inspect background work	Agents or commands are running
`/agents`	Manage subagents	Working with delegated agents
`/background`	Detach current session	Claude can keep working without this terminal
`/batch`	Split large repo-wide work	Migration or large mechanical refactor
`/mcp`	Manage MCP integrations	Connect external tools/data
`/hooks`	Inspect configured hooks	Automatic workflow enforcement
`/branch`	Branch the conversation	Try another approach safely

Anthropic's current command reference groups these around setup, active development, parallel work, review, session management, and troubleshooting.

03

/clear vs /compact vs /branch

These solve three different problems.

/clear	/compact	/branch
Different problem	Same problem + too much context	Try another approach safely

```
/clear
```

USE IT WHEN

You're moving from one unrelated task to another.

EXAMPLE

You just spent 40 minutes debugging authentication. Now you want to redesign the product page.

Run `/clear` because all that authentication context probably isn't useful anymore.

```
/compact
```

USE IT WHEN

You're still working on the same problem, but the session has become large.

You can tell Claude what matters:

```
/compact preserve all database schema decisions, migration problems and unresolved inventory issues
```

So:

Same problem + too much context = `/compact`

Different problem = `/clear`

```
/branch try-different-schema
```

This creates another conversation from the current point while leaving the original intact.

USE IT WHEN

You want to experiment with another direction without destroying the path you're already on.

EXAMPLE

You've spent an hour designing inventory around:

CURRENT PATH	BRANCH
stock stored directly on Product	a separate inventory table

but you want to see whether a separate inventory table would be cleaner.

Instead of derailing your current conversation:

```
/branch inventory-table-experiment
```

Then explore the alternative there. You can later resume the original session.

04

/fork

This one deserves its own explanation because its behavior has changed.

In standard Claude Code behavior, `/fork` can act as an alias for `/branch`.

That means:

```
/fork experimental-checkout
```

can branch the current conversation.

FORKED SUBAGENT BEHAVIOR

Claude Code also has a newer forked subagent behavior. When that functionality is enabled, `/fork` can instead launch another agent that inherits the current conversation context and works in the background.

EXAMPLE

```
/fork draft unit tests for the parser changes so far
```

You can keep working in the main conversation while the fork handles that side task.

This is different from a normal subagent because a normal subagent starts with a focused context supplied by the parent, while a fork can inherit the current conversation history.

NORMAL SUBAGENT	FORK
starts with a focused context supplied by the parent	can inherit the current conversation history

USE FORK WHEN

The side task needs to understand a lot of what has already happened in the current conversation.

EXAMPLE

MAIN AGENT	Implementing checkout.
FORK	`/fork review everything we've decided about checkout so far and identify edge cases we haven't covered`

Meanwhile, you keep implementing.

Context Management

Claude doesn't only carry your messages.

The context window can contain:

conversation history	tool results
files Claude read	`CLAUDE.md`
terminal output	memory
loaded Skills	MCP results
information returned from subagents	

- That means context quality matters.

```
/context
```

Use it when Claude seems to be:

- forgetting instructions
- repeating itself
- referencing stale decisions
- becoming less focused
- carrying too much irrelevant information

Think of `/context` as:

> What's currently taking up Claude's attention?

`/btw`

For a small side question:

```
/btw Why did you use a composite index here?
```

This is useful when you want an answer without adding that entire tangent to your main conversation history.

CLAUDE.md

CLAUDE.md is persistent project guidance Claude can load when working in your project.

A project-level file might contain:

CLAUDE.MD — PROJECT LEVEL

```
# Commands
- Package manager: pnpm
- Dev server: `pnpm dev`
- Typecheck: `pnpm typecheck`
- Tests: `pnpm test`

# Database
- Use Drizzle ORM for database access
- Schema lives inside `src/db/schema/`
- Never manually modify generated migration files

# Frontend
- Reuse existing UI primitives before creating new ones
- Follow the established spacing and typography tokens
```

/init

```
/init
```

Claude inspects the repository and creates or improves a starting `CLAUDE.md` .

USE IT WHEN

You open Claude Code inside:

- an existing repository
- a project you didn't build
- a legacy app
- a monorepo
- a new project that has started developing conventions

Don't blindly keep everything Claude generates.

Review it.

07

Where Instructions Should Live

A simple rule:

Something Claude should always know	→ `CLAUDE.md`
A repeatable way of doing work	→ `Skill`
Something that MUST happen	→ `Hook`
Something Claude must not be allowed to do	→ `Permissions`
A one-time request	→ `Prompt`
External service or tool	→ `MCP`
Separate focused investigation	→ `Subagent`

This distinction becomes very important as your setup grows.

Permissions

Claude Code isn't just producing text.

It can actually:

`Read` `Write` `Edit` `Run Bash` `Use Git` `Call MCP tools`

`Access external services`

Permissions control which of those actions are allowed.

Open them using:

```
/permissions
```

The core rule groups are:

`allow` `ask` `deny`

EXAMPLE

SETTINGS

```
{
  "permissions": {
    "allow": [
      "Bash(pnpm typecheck)",
      "Bash(pnpm test *)",
      "Bash(git status)",
      "Bash(git diff *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(.env)",
      "Read(.env.*)",
      "Edit(.env)",
      "Edit(.env.*)"
    ]
  }
}
```

Good use

LET CLAUDE FREELY RUN — pnpm typecheck · pnpm test · git status

BUT ASK BEFORE — git push

AND PREVENT DIRECT ACCESS TO — .env

09

Permission Modes

Press:

`Shift` + `Tab`

to cycle between available modes.

You'll commonly work with:

`Manual`	Claude asks more often. Good while learning or working somewhere sensitive.
`Accept Edits`	Normal edits become less annoying to approve. Useful once you're comfortable with the task.
`Plan`	Claude investigates and proposes first.
`Auto`	Claude Code uses its safety system to decide whether many actions can proceed automatically.

Always look at the current mode rather than assuming you're in the same mode as someone else's tutorial.

10

Plan Mode

Enter it with:

```
/plan
```

EXAMPLE

```
/plan Replace the current local product data with PostgreSQL using
Drizzle. Study how products are currently consumed before deciding the
schema. We only need categories, products and stock right now.
```

Plan Mode is for:

01	02	03
Think first.	Inspect first.	Don't jump straight into implementation.

Claude can explore the project and form an implementation approach before you approve normal source edits.

Good tasks for Plan Mode

Use it for:

database changes	authentication
permissions	architecture
migration work	large refactors
unfamiliar code	features spanning several systems
decisions that are expensive to reverse	

Usually skip it for

Change this text	Fix this margin
Rename this variable	Add an icon

EXAMPLE

BAD Build product variants.

BETTER

```
/plan Study the current product and inventory implementation and
propose how product variants should be introduced without breaking
existing products. Include schema changes, migration path, API impact,
admin changes and storefront changes. Don't implement yet.
```

Then challenge the plan:

- → Why do we need a separate inventory table?
- → Can this first version avoid variant-level pricing?
- → What existing code will break if we do it this way?

That conversation can save you from reviewing a terrible 2,000-line diff later.

Models

Open:

```
/model
```

Instead of memorizing today's model list, choose based on the job.

Faster model Good for:	Stronger reasoning model Good for:
Simple edits	Architecture
Renaming	Subtle debugging
Formatting	Database design
Routine CRUD	Security review
Basic tests	Large refactors
Straightforward components	Deep code review
	Unfamiliar legacy code

Effort

Use:

```
/effort
```

or:

```
/effort high
```

Think about model and effort separately.

Model	Effort
which brain you're using	how hard you want it to think on this task

DON'T USE MAXIMUM EFFORT FOR

Change button text to "Checkout"

USE HIGHER EFFORT FOR

Figure out why checkout occasionally creates duplicate orders under concurrent requests.

Skills

A Skill is a reusable process Claude can follow.

Project Skills live inside:

```
.claude/skills/
```

EXAMPLE

PROJECT STRUCTURE

```
.claude/
  skills/
    build-ui/
      SKILL.md
```

Example `/build-ui`

SKILL.MD

```
---
description: Build or significantly update customer-facing UI
using the existing project's components, design system and re
sponsive patterns.
---

Build:

$ARGUMENTS
```

BEFORE IMPLEMENTING	DURING IMPLEMENTATION	BEFORE FINISHING
1. Inspect the existing page.	1. Reuse existing primitives.	1. Run the application.
2. Search for reusable components.	2. Keep the change focused.	2. Inspect the result.
3. Inspect the design tokens.	3. Follow established typography and spacing.	3. Run typecheck.
4. Check how similar pages are structured.	4. Make it responsive.	4. Report what changed.

Now:

```
/build-ui create the New Arrivals page
```

When to Create a Skill

If you catch yourself repeatedly typing something like:

First inspect this.
Then check that.
Then follow these conventions.
Then test these things.
Then report using this structure.

that's probably a Skill.

Useful examples:

`/build-ui` `/review-ui` `/build-feature` `/verify-feature`

`/review-changes` `/debug` `/deploy`

Dynamic Skills

Skills can include live command output.

EXAMPLE

SKILL.MD

```
## Current Diff!`git diff HEAD`

Review these changes for:

- bugs
- regressions
- unnecessary changes
- missing tests
- violations of CLAUDE.md
```

Then:

```
/review-changes
```

automatically reviews whatever currently changed.

> No copying the diff manually.

Subagents

A subagent is another Claude worker with its own context.

> The important part isn't that it's an “AI expert.”

The important part is:

01	02	03
Separate context.	Focused task.	Useful result returned to the main agent.

Good use

Suppose you're implementing inventory management.

The main Claude could delegate:

MAIN CLAUDE

```
Agent 1  →  inspect database and stock model
Agent 2  →  inspect admin product editing flow
Agent 3  →  inspect test coverage and edge cases
```

Each agent can read lots of files.

The main conversation only needs their findings.

/agents

Current Claude Code docs again show `/agents` as the interface for managing subagents, including active agents and reusable agent configurations. Older Claude Code versions changed this behavior several times, so if an older tutorial says `/agents` disappeared, check your current version.

Custom subagent

Create:

```
.claude/agents/feature-reviewer.md
```

EXAMPLE

FEATURE-REVIEWER.MD

```
---
name: feature-reviewer
description: Review a completed feature in a fresh context fo
r bugs, regressions, unnecessary changes and missing verifica
tion.
tools: Read, Grep, Glob
---

Review the requested feature.

Return:

## Critical## Major## Minor## Verification## Recommendation
```

Then:

```
→ Use the feature-reviewer agent to independently review the inventory feature.
```

Custom agents can also control things such as model, effort, permissions, Skills, MCP servers, background execution and worktree isolation.

17

/tasks

```
/tasks
```

Shows work currently running in the background of this session.

Use it when:

- Claude started subagents.
- A Bash command is still running.
- A fork is working.
- Something has been backgrounded.

You can inspect and manage those running tasks from there.

/background

This is where Claude Code gets much more interesting.

Run: `/background` or: `/bg`

to detach the current Claude session and let it continue working without keeping your terminal occupied.

You can also give it one last instruction:

```
/bg run the full test suite and fix any failures
```

Claude keeps working in the background.

You can then manage those sessions using:

```
claude agents
```

Anthropic's current CLI also lets you start directly in the background:

```
claude --bg "investigate the flaky checkout test"
```

And later:

```
claude attach <id>
```

```
claude logs <id>
```

```
claude stop <id>
```

Agent View

Run:

```
claude agents
```

This gives you a screen where you can see multiple background Claude sessions and which ones are working, need input, finished, failed.

This is different from:

```
/agents
```

Think of them like this:

`/agents`	`claude agents`
→ agents inside the current Claude workflow	→ dashboard for multiple full Claude sessions

Anthropic currently describes Agent View as a research preview.

Git Worktrees

If two agents are editing the same checkout at once, you can get conflicts fast.

Worktrees solve the filesystem side of that problem.

Start Claude inside an isolated worktree:

```
claude --worktree feature-auth
```

or:

```
claude -w feature-auth
```

Claude creates an isolated checkout under:

```
.claude/worktrees/
```

by default.

EXAMPLE

TERMINAL 1	TERMINAL 2	TERMINAL 3
`claude -w inventory`	`claude -w checkout`	`claude -w admin-dashboard`

Now three Claude sessions can work without directly editing the same filesystem checkout.

Worktree vs Subagent

These solve different problems.

Subagent	Worktree
→ context isolation	→ filesystem isolation

You can combine both.

A custom agent can include:

```
isolation: worktree
```

Then whenever that subagent changes code, it gets its own isolated checkout.

/batch

This is for big changes.

EXAMPLE

```
/batch migrate src/ from Solid to React
```

Claude first researches the codebase and breaks the job into independent units.

01	Claude researches the codebase
02	splits a large change into roughly 5-30 independent pieces
03	you approve the plan
04	units are handed to background subagents in isolated worktrees
05	each unit can implement, test, and open a pull request

Current Anthropic docs say `/batch` can split a large change into roughly 5-30 independent pieces. After you approve the plan, those units are handed to background subagents in isolated worktrees, and each unit can implement, test, and open a pull request.

Good `/batch` tasks

```
/batch migrate all Jest tests to Vitest
```

```
/batch replace deprecated API client usage across packages
```

```
/batch move all legacy React class components in src/ to functional components
```

```
/batch update every package in this monorepo to the new logging API
```

These are large, divisible changes.

Bad `/batch` tasks

```
/batch build checkout
```

Checkout may involve tightly coupled decisions.

Not every large feature can safely be sliced into independent PRs.

> Use `/batch` when the work is genuinely separable.

MCP

MCP stands for Model Context Protocol.

It lets Claude connect to tools and systems outside your local codebase.

THINK

GitHub | Jira | Sentry | Notion | Databases

Internal APIs | Monitoring systems | Custom business tools

Anthropic's MCP docs recommend using MCP when you find yourself repeatedly copying information from another service into Claude manually.

Add an MCP server

EXAMPLE

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

LIST SERVERS	`claude mcp list`
INSPECT ONE	`claude mcp get notion`
REMOVE ONE	`claude mcp remove notion`

Inside Claude Code:

```
/mcp
```

opens the MCP manager and can also handle authentication for supported remote servers.

Hooks

A SKILL SAYS	A HOOK SAYS
Claude should follow this workflow.	This action runs automatically when this event happens.

That's a major difference.

Anthropic describes hooks as deterministic automation tied to events in Claude Code's lifecycle.

/hooks

```
/hooks
```

opens the configured hooks browser.

Current versions treat this mainly as an inspection interface. You configure hooks in settings files or ask Claude to create them.

Hook Example: Format Files After Claude Edits Them

SETTINGS POSTTOOLUSE

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write ."
          }
        ]
      }
    ]
  }
}
```

Now formatting doesn't depend on Claude remembering:

> Please run Prettier.

It happens automatically.

Hook Example: Block Dangerous Commands

Use:

```
PreToolUse
```

before Bash commands.

Your script can inspect the command and reject things such as:

```
rm -rf /
```

```
DROP DATABASE
```

```
git push --force
```

This is stronger than adding:

> Don't run dangerous commands.

to a prompt.

Important Hook Events

Useful ones include:

`SessionStart`	`TeammateIdle`
`UserPromptSubmit`	`PreCompact`
`PreToolUse`	`PostCompact`
`PostToolUse`	`WorktreeCreate`
`Notification`	`WorktreeRemove`
`SubagentStart`	`Stop`
`SubagentStop`	`SessionEnd`
`TaskCreated`	
`TaskCompleted`	

Current Claude Code exposes many lifecycle events, including specific events for subagents, agent teams, tasks, and worktrees.

Skills vs Hooks vs MCP vs Subagents

This is worth remembering:

`CLAUDE.md`	→ permanent project context
`Skills`	→ repeatable way of doing work
`Hooks`	→ something that should happen automatically
`MCP`	→ connection to another system
`Subagents`	→ isolated delegated work
`Worktrees`	→ isolated files
`Agent teams`	→ multiple agents coordinating together

Useful Keyboard Shortcuts

SHORTCUT	USE
`Ctrl+C`	Interrupt Claude
`Esc`	Stop current work
`Esc Esc`	Open checkpoint/rewind controls
`Shift+Tab`	Cycle permission modes
`Ctrl+G`	Open content in external editor
`Ctrl+R`	Search command/prompt history
`Ctrl+B`	Background supported running work
`Ctrl+T`	Toggle task list
`/`	Browse commands and Skills
`@`	Mention files/directories
`!`	Run a shell command

34

Mention Files Directly

Instead of:

```
Look somewhere in the authentication code.
```

you can say:

```
Review @src/auth/session.ts
```

Or:

```
Compare @src/db/schema.ts with @src/server/inventory.ts
```

Useful when you already know what matters.

35

Shell Commands

You can run shell commands directly from Claude Code with:

```
!git status
```

or:

```
!pnpm test
```

Useful when you want the result inside the current conversation without asking Claude to decide whether to run it.

/doctor

If Claude Code behaves strangely:

```
/doctor
```

or:

```
claude doctor
```

Use it for:

Installation problems	Hook problems
Broken settings	Plugin problems
MCP issues	Environment issues

A Practical Everyday Workflow

For a normal feature:

01	Start Claude
02	Make sure project context is correct
03	Give it one focused feature
04	Use `/plan` if the approach matters
05	Review the plan
06	Implement
07	Watch what files and commands Claude uses
08	Let Claude verify the result
09	`/diff`
10	Run tests
11	Use a fresh subagent or `/review` for important changes
12	`/security-review` when appropriate
13	Update `CLAUDE.md` if a permanent project rule changed
14	`/compact` if you're continuing the same work
15	`/clear` before unrelated work

Quick Decision Guide

Claude doesn't understand the project	`CLAUDE.md` `/init`
Claude keeps forgetting something discovered during previous sessions	`/memory`
Current session is getting messy	`/context`
Same task, context too large	`/compact`
Completely different task	`/clear`
Need to think before coding	`/plan`
Want another approach without losing this conversation	`/branch`
Side task needs current conversation context	`/fork`
Repeated workflow	`Skill`
Something must happen automatically	`Hook`

Need Jira/GitHub/Sentry/database/etc.	`MCP`
Focused investigation with isolated context	`Subagent`
Need multiple agents coordinating	`Agent Team`
Need filesystem isolation	`Worktree`
Huge migration that splits into independent pieces	`/batch`
Want current Claude session to continue elsewhere	`/background`
Want to inspect background work	`/tasks` `claude agents`
Claude went in the wrong direction	`/rewind`

Ready to ship	`/diff`
	`/review`
	`/security-review`

Something is broken in Claude Code itself	`/doctor`

ONE MENTAL MODEL TO REMEMBER

You don't get better results from Claude Code by memorizing more commands. You get better results by knowing which problem you're trying to solve.

Need context? → `CLAUDE.md`	Need isolated files? → `Worktree`
Need a repeatable process? → `Skill`	Need coordinated workers? → `Agent Team`
Need a guaranteed action? → `Hook`	Need large-scale parallel work? → `/batch`
Need an external system? → `MCP`	Need Claude working while you move on? → `/background`
Need isolated thinking? → `Subagent`	Need to review the approach first? → `/plan`
Need to recover from a mistake? → `/rewind`	

Once you understand those boundaries, Claude Code stops feeling like a giant collection of features and starts feeling like a set of engineering controls you can combine depending on the work you're doing.

NEXT LEVEL

Agentic Engineering

If you're comfortable with everything in this cheat sheet, the next level isn't learning another 100 Claude Code commands.

> **It's learning how to combine these ideas into a repeatable engineering process.**

That's what I go much deeper into inside my Agentic Engineering course.

The course covers how to use Claude Code and other AI coding tools across real production applications, larger codebases, monorepos and legacy projects, including:

scoping work before implementation	independent review	Git worktrees
codebase auditing	testing	MCP
architecture decisions	documentation	hooks
feature-by-feature development	keeping context synchronized	parallel development
verification	multiple agents	production workflows

I've also made the underlying Agentic Engineering Skills completely free and open source, so you can use the workflow yourself.

The course is where I show you how all of those parts fit together when the project becomes much bigger than a demo.